Inline Functions

- Instead of calling a function, compiler replaces the code at the function call point.
- Keyword **‘inline’** is used to request compiler to make a function inline. It is a request and not a command.
- If we define the function inside the class body then the function is by default an inline function.
- In case function is defined outside the class body then we must use the keyword **‘inline’** to make a function inline.

Inline Functions

- inline int Area(int len, int hi)
- {
- return len * hi;
- }
- int main()
- {
- cout << Area(10,20);
- }

After Compilation the code in main () will look like:

- int main()
- {
- cout << len * hi;
- **OR**
- cout << 10 * 20;
- }

Example of inline function.

- class Student{
- private:
- int rollNo;
- public:
- void setRollNo(int aRollNo){
- ***//* by default it is inline because it is defined inside the class body.**
- rollNo = aRollNo;
- }
- };

Another example with functions defined outside the class body:

- class Student{
- private:
- … public:
- inline void setRollNo(int aRollNo);
- };
- void Student**::**setRollNo(int aRollNo){
- // no need to put inline in definition, as it is declared in the class as inline.
- rollNo = aRollNo;
- }

Another example without using inline keyword in functions definition:

- class Student{
- private:
- …
- public:
- void setRollNo(int aRollNo);
- };
- inline void Student::setRollNo(int aRollNo){
- **//keyword place in the start of signature**
- rollNo = aRollNo;
- }

- **OR**
- void inline Student::setRollNo(int aRollNo){
- **//keyword place after function return type**
- rollNo = aRollNo;
- }

Another example of defining an inline function where the keyword is place both in declaration and definition:

- class Student{
- private:
- …
- public:
- inline void setRollNo(int aRollNo);
- };
- inline void Student::setRollNo(int aRollNo){
- rollNo = aRollNo;
- }

Advantages of Inline Functions

1) Function call overhead doesn’t occur
2) It also saves overhead of a return call from a function
3) Inline function may be useful (if it is small) for embedded systems because inline can yield less code than the function call preamble and return

Disadvantages of Inline Functions

1) The added variables from the inline function consumes additional registers

2) If you use too many inline functions then the size of the binary executable file will be large, because of the duplication of same code

3) Inline function may increase compile time overhead if someone changes the code inside the inline function then all the calling location has to be recompiled

4) Inline functions may not be useful for many embedded systems. Because in embedded systems code size is more important than speed

Inline Functions

- Compiler **does not** perform inlining when:

1) If a function contains a loop
2) If a function contains static variables
3) If a function is recursive
4) If a function return type is other than void, and the return statement doesn’t exist in function body
5) If a function contains switch or goto statement